
chapter

5

BASIC PROCESSING UNIT

CHAPTER OBJECTIVES

In this chapter you will learn about:

- Execution of instructions by a processor
- The functional units of a processor and how they are interconnected
- Hardware for generating control signals
- Microprogrammed control

In this chapter we focus on the processing unit, which executes machine-language instructions and coordinates the activities of other units in a computer. We examine its internal structure and show how it performs the tasks of fetching, decoding, and executing such instructions. The processing unit is often called the *central processing unit* (CPU). The term “central” is not as appropriate today as it was in the past, because today’s computers often include several processing units. We will use the term *processor* in this discussion.

The organization of processors has evolved over the years, driven by developments in technology and the desire to provide high performance. To achieve high performance, it is prudent to make various functional units of a processor operate in parallel as much as possible. Such processors have a *pipelined* organization where the execution of an instruction is started before the execution of the preceding instruction is completed. Another approach, known as *superscalar* operation, is to fetch and start the execution of several instructions at the same time. Pipelining and superscalar approaches are discussed in Chapter 6. In this chapter, we concentrate on the basic ideas that are common to all processors.

5.1 SOME FUNDAMENTAL CONCEPTS

A typical computing task consists of a series of operations specified by a sequence of machine-language instructions that constitute a program. The processor fetches one instruction at a time and performs the operation specified. Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered. The processor uses the *program counter*, PC, to keep track of the address of the next instruction to be fetched and executed. After fetching an instruction, the contents of the PC are updated to point to the next instruction in sequence. A branch instruction may cause a different value to be loaded into the PC.

When an instruction is fetched, it is placed in the *instruction register*, IR, from where it is interpreted, or decoded, by the processor’s control circuitry. The IR holds the instruction until its execution is completed.

Consider a 32-bit computer in which each instruction is contained in one word in the memory, as in RISC-style instruction set architecture. To execute an instruction, the processor has to perform the following steps:

1. Fetch the contents of the memory location pointed to by the PC. The contents of this location are the instruction to be executed; hence they are loaded into the IR. In register transfer notation, the required action is

$$\text{IR} \leftarrow [\text{PC}]$$

2. Increment the PC to point to the next instruction. Assuming that the memory is byte addressable, the PC is incremented by 4; that is

$$\text{PC} \leftarrow [\text{PC}] + 4$$

3. Carry out the operation specified by the instruction in the IR.

Fetching an instruction and loading it into the IR is usually referred to as the *instruction fetch phase*. Performing the operation specified in the instruction constitutes the *instruction execution phase*.

With few exceptions, the operation specified by an instruction can be carried out by performing one or more of the following actions:

- Read the contents of a given memory location and load them into a processor register.
- Read data from one or more processor registers.
- Perform an arithmetic or logic operation and place the result into a processor register.
- Store data from a processor register into a given memory location.

The hardware components needed to perform these actions are shown in Figure 5.1. The processor communicates with the memory through the processor-memory interface, which transfers data from and to the memory during Read and Write operations. The instruction address generator updates the contents of the PC after every instruction is fetched. The register file is a memory unit whose storage locations are organized to form the processor's general-purpose registers. During execution, the contents of the registers named in an instruction that performs an arithmetic or logic operation are sent to the arithmetic and logic

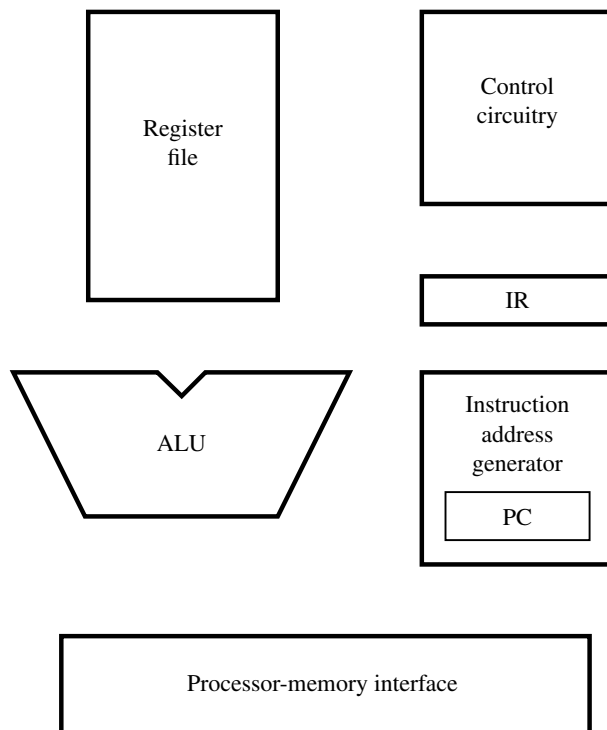


Figure 5.1 Main hardware components of a processor.

unit (ALU), which performs the required computation. The results of the computation are stored in a register in the register file.

Before we examine these units and their interaction in detail, it is helpful to consider the general structure of any data processing system.

Data Processing Hardware

A typical computation operates on data stored in registers. These data are processed by combinational circuits, such as adders, and the results are placed into a register. Figure 5.2 illustrates this structure. A clock signal is used to control the timing of data transfers. The registers comprise edge-triggered flip-flops into which new data are loaded at the active edge of the clock. In this chapter, we assume that the rising edge of the clock is the active edge. The clock period, which is the time between two successive rising edges, must be long enough to allow the combinational circuit to produce the correct result.

The operation performed by the combinational block in Figure 5.2 may be quite complex. It can often be broken down into several simpler steps, where each step is performed by a subcircuit of the original circuit. These subcircuits can then be cascaded into a multi-stage structure as shown in Figure 5.3. Then, if n stages are used, the operation will be completed in n clock cycles. Since these combinational subcircuits are smaller, they can complete their operation in less time, and hence a shorter clock period can be used. A key advantage of the multi-stage structure is that it is suitable for pipelined operation, as will be discussed in Chapter 6. Such a structure is particularly useful for implementing processors that have a RISC-style instruction set. The discussion in the remainder of this chapter focuses on processors that use a multi-stage structure of this type. In Section 5.7 we will consider a more traditional alternative that is suitable for CISC-style processors.

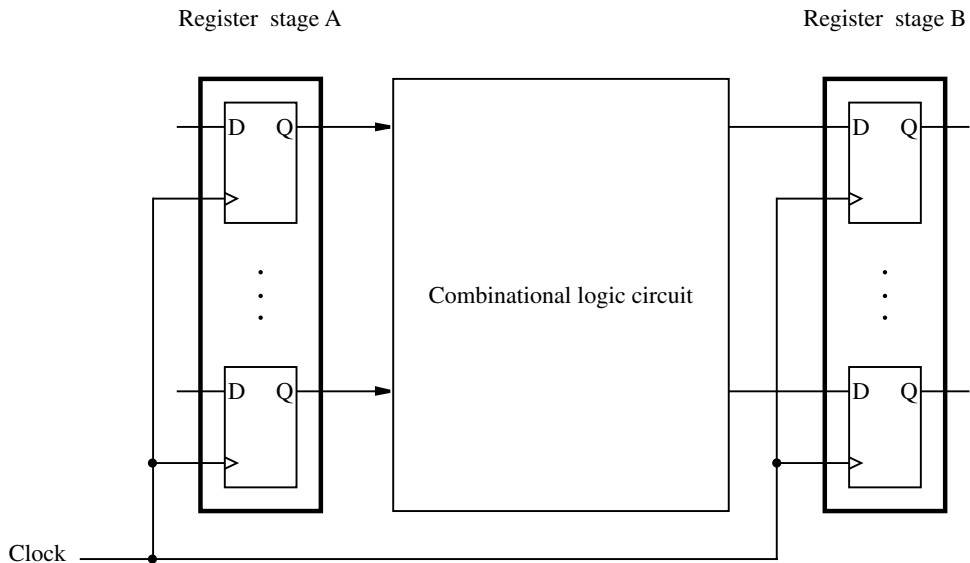


Figure 5.2 Basic structure for data processing.

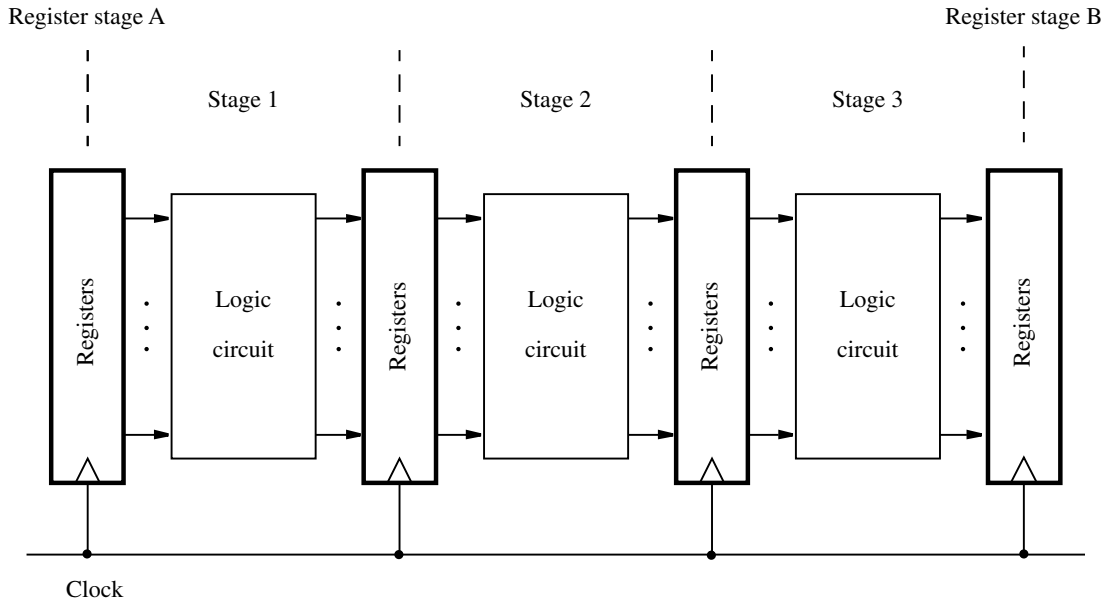


Figure 5.3 A hardware structure with multiple stages.

5.2 INSTRUCTION EXECUTION

Let us now examine the actions involved in fetching and executing instructions. We illustrate these actions using a few representative RISC-style instructions.

5.2.1 LOAD INSTRUCTIONS

Consider the instruction

Load R5, X(R7)

which uses the Index addressing mode to load a word of data from memory location $X + [R7]$ into register R5. Execution of this instruction involves the following actions:

- Fetch the instruction from the memory.
- Increment the program counter.
- Decode the instruction to determine the operation to be performed.
- Read register R7.
- Add the immediate value X to the contents of R7.
- Use the sum $X + [R7]$ as the effective address of the source operand, and read the contents of that location in the memory.
- Load the data received from the memory into the destination register, R5.

Depending on how the hardware is organized, some of these actions can be performed at the same time. In the discussion that follows, we will assume that the processor has five hardware stages, which is a commonly used arrangement in RISC-style processors. Execution of each instruction is divided into five steps, such that each step is carried out by one hardware stage. In this case, fetching and executing the Load instruction above can be completed as follows:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read the contents of register R7 in the register file.
3. Compute the effective address.
4. Read the memory source operand.
5. Load the operand into the destination register, R5.

5.2.2 ARITHMETIC AND LOGIC INSTRUCTIONS

Instructions that involve an arithmetic or logic operation can be executed using similar steps. They differ from the Load instruction in two ways:

- There are either two source registers, or a source register and an immediate source operand.
- No access to memory operands is required.

A typical instruction of this type is

Add R3, R4, R5

It requires the following steps:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read the contents of source registers R4 and R5.
3. Compute the sum $[R4] + [R5]$.
4. Load the result into the destination register, R3.

The Add instruction does not require access to an operand in the memory, and therefore could be completed in four steps instead of the five steps needed for the Load instruction. However, as we will see in the next chapter, it is advantageous to use the same multi-stage processing hardware for as many instructions as possible. This can be achieved if we arrange for all instructions to be executed in the same number of steps. To this end, the Add instruction should be extended to five steps, patterned along the steps of the Load instruction. Since no access to memory operands is required, we can insert a step in which no action takes place between steps 3 and 4 above. The Add instruction would then be performed as follows:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R4 and R5.
3. Compute the sum $[R4] + [R5]$.

4. No action.
5. Load the result into the destination register, R3.

If the instruction uses an immediate operand, as in

Add R3, R4, #1000

the immediate value is given in the instruction word. Once the instruction is loaded into the IR, the immediate value is available for use in the addition operation. The same five-step sequence can be used, with steps 2 and 3 modified as:

2. Decode the instruction and read register R4.
3. Compute the sum $[R4] + 1000$.

5.2.3 STORE INSTRUCTIONS

The five-step sequence used for the Load and Add instructions is also suitable for Store instructions, except that the final step of loading the result into a destination register is not required. The hardware stage responsible for this step takes no action. For example, the instruction

Store R6, X(R8)

stores the contents of register R6 into memory location $X + [R8]$. It can be implemented as follows:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R6 and R8.
3. Compute the effective address $X + [R8]$.
4. Store the contents of register R6 into memory location $X + [R8]$.
5. No action.

After reading register R8 in step 2, the memory address is computed in step 3 using the immediate value, X, in the IR. In step 4, the contents of R6 are sent to the memory to be stored. No action is taken in step 5.

In summary, the five-step sequence of actions given in Figure 5.4 is suitable for all instructions in a RISC-style instruction set. RISC-style instructions are one word long and only Load and Store instructions access operands in the memory, as explained in Chapter 2. Instructions that perform computations use data that are either stored in general-purpose registers or given as immediate data in the instruction.

The five-step sequence is suitable for all Load and Store instructions, because the addressing modes that can be used in these instructions are special cases of the Index mode. Most RISC-style processors provide one general-purpose register, usually register R0, that always contains the value zero. When R0 is used as the index register, the effective address of the operand is the immediate value X. This is the Absolute addressing mode. Alternatively, if the offset X is set to zero, the effective address is the contents of the index register, R_i . This is the Indirect addressing mode. Thus, only one addressing mode, the Index mode,

Step	Action
1	Fetch an instruction and increment the program counter.
2	Decode the instruction and read registers from the register file.
3	Perform an ALU operation.
4	Read or write memory data if the instruction involves a memory operand.
5	Write the result into the destination register, if needed.

Figure 5.4 A five-step sequence of actions to fetch and execute an instruction.

needs to be implemented, resulting in a significant simplification of the processor hardware. The task of selecting R0 as the index register or setting X to zero is left to the assembler or the compiler. This is consistent with the RISC philosophy of aiming for simple and fast hardware at the expense of higher compiler complexity and longer compilation time. The result is a net gain in the time needed to perform various tasks on a computer, because programs are compiled much less frequently than they are executed.

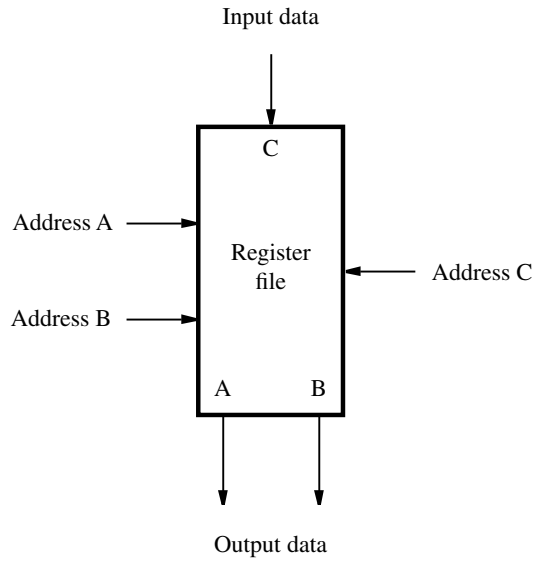
5.3 HARDWARE COMPONENTS

The discussion above indicates that all instructions of a RISC-style processor can be executed using the five-step sequence in Figure 5.4. Hence, the processor hardware may be organized in five stages, such that each stage performs the actions needed in one of the steps. We now examine the components in Figure 5.1 to see how they may be organized in the multi-stage structure of Figure 5.3.

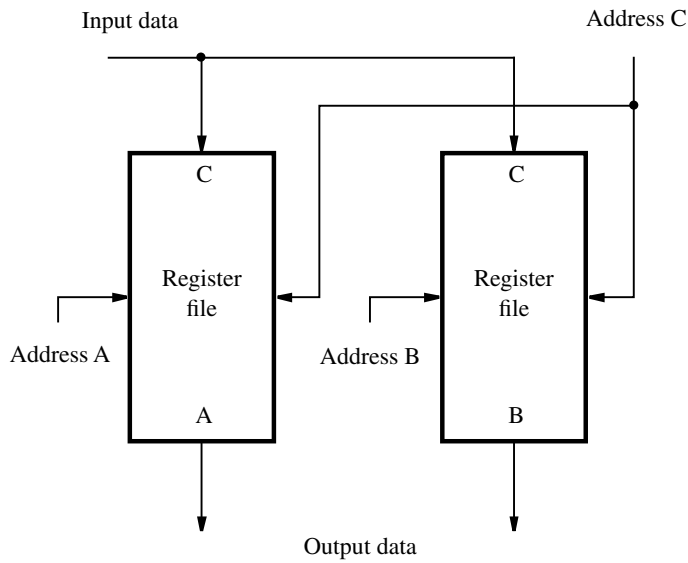
5.3.1 REGISTER FILE

General-purpose registers are usually implemented in the form of a register file, which is a small and fast memory block. It consists of an array of storage elements, with access circuitry that enables data to be read from or written into any register. The access circuitry is designed to enable two registers to be read at the same time, making their contents available at two separate outputs, A and B. The register file has two address inputs that select the two registers to be read. These inputs are connected to the fields in the IR that specify the source registers, so that the required registers can be read. The register file also has a data input, C, and a corresponding address input to select the register into which data are to be written. This address input is connected to the IR field that specifies the destination register of the instruction.

The inputs and outputs of any memory unit are often called input and output *ports*. A memory unit that has two output ports is said to be *dual-ported*. Figure 5.5 shows two ways



(a) Single memory block



(b) Two memory blocks

Figure 5.5 Two alternatives for implementing a dual-ported register file.

of realizing a dual-ported register file. One possibility is to use a single set of registers with duplicate data paths and access circuitry that enable two registers to be read at the same time. An alternative is to use two memory blocks, each containing one copy of the register file. Whenever data are written into a register, they are written into both copies of that register. Thus, the two files have identical contents. When an instruction requires data from two registers, one register is accessed in each file. In effect, the two register files together function as a single dual-ported register file.

5.3.2 ALU

The arithmetic and logic unit is used to manipulate data. It performs arithmetic operations such as addition and subtraction, and logic operations such as AND, OR, and XOR. Conceptually, the register file and the ALU may be connected as shown in Figure 5.6. When an instruction that performs an arithmetic or logic operation is being executed, the contents of the two registers specified in the instruction are read from the register file and become

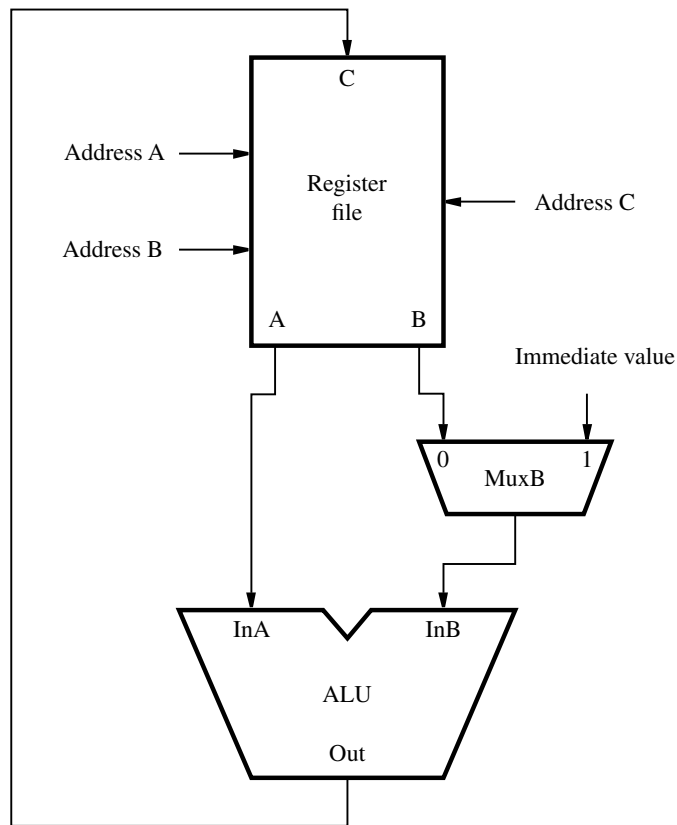


Figure 5.6 Conceptual view of the hardware needed for computation.

available at outputs A and B. Output A is connected directly to the first input of the ALU, InA, and output B is connected to a multiplexer, MuxB. The multiplexer selects either output B of the register file or the immediate value in the IR to be connected to the second ALU input, InB. The output of the ALU is connected to the data input, C, of the register file so that the results of a computation can be loaded into the destination register.

5.3.3 DATAPATH

Instruction processing consists of two phases: the fetch phase and the execution phase. It is convenient to divide the processor hardware into two corresponding sections. One section fetches instructions and the other executes them. The section that fetches instructions is also responsible for decoding them and for generating the control signals that cause appropriate actions to take place in the execution section. The execution section reads the data operands specified in an instruction, performs the required computations, and stores the results.

We now need to organize the hardware into a multi-stage structure similar to that in Figure 5.3, with stages corresponding to the five steps in Figure 5.4. A possible structure is shown in Figure 5.7. The actions taken in each of the five stages are completed in one clock cycle. An instruction is fetched in step 1 by hardware stage 1 and placed into the IR. It is decoded, and its source registers are read in step 2. The information in the IR is used to generate the control signals for all subsequent steps. Therefore, the IR must continue to hold the instruction until its execution is completed.

It is necessary to insert registers between stages. Inter-stage registers hold the results produced in one stage so that they can be used as inputs to the next stage during the next clock cycle. This leads to the organization in Figure 5.8. The hardware in the figure is often referred to as the *datapath*. It corresponds to stages 2 to 5 in Figure 5.7. Data read from the register file are placed in registers RA and RB. Register RA provides the data to input InA of the ALU. Multiplexer MuxB forwards either the contents of RB or the immediate value in the IR to the ALU's second input, InB. The ALU constitutes stage 3, and the result of the computation it performs is placed in register RZ.

Recall that for computational instructions, such as an Add instruction, no processing actions take place in step 4. During that step, multiplexer MuxY in Figure 5.8 selects register RZ to transfer the result of the computation to RY. The contents of RY are transferred to the register file in step 5 and loaded into the destination register. For this reason, the register file is in both stages 2 and 5. It is a part of stage 2 because it contains the source registers and a part of stage 5 because it contains the destination register.

For Load and Store instructions, the effective address of the memory operand is computed by the ALU in step 3 and loaded into register RZ. From there, it is sent to the memory, which is stage 4. In the case of a Load instruction, the data read from the memory are selected by multiplexer MuxY and placed in register RY, to be transferred to the register file in the next clock cycle. For a Store instruction, data are read from the register file, which is part of stage 2, and placed in register RB. Since memory access is done in stage 4, another inter-stage register is needed to maintain correct data flow in the multi-stage structure. Register RM is introduced for this purpose. The data to be stored are moved from RB to RM in step 3, and from there to the memory in step 4. No action is taken in step 5 in this case.

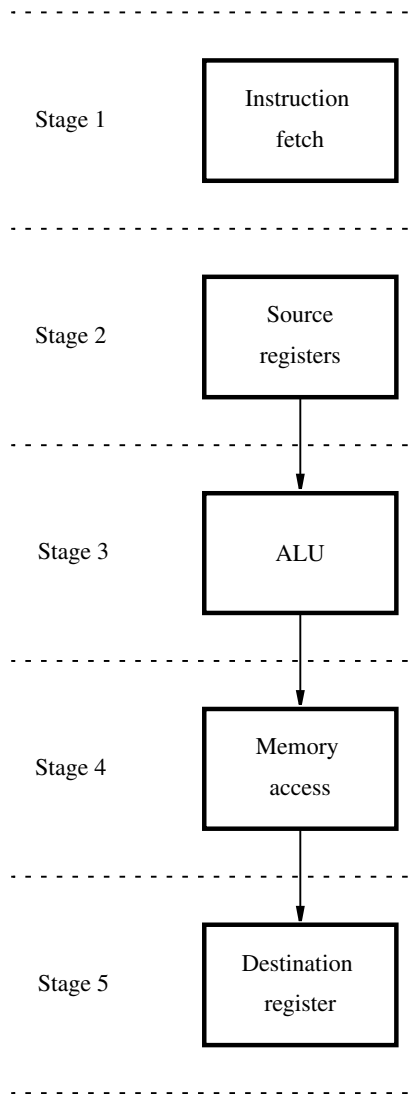


Figure 5.7 A five-stage organization.

The subroutine call instructions introduced in Section 2.7 save the return address in a general-purpose register, which we call LINK for ease of reference. Similarly, interrupt processing requires a return address to be saved, as described in Section 3.2. Assume that another general-purpose register, IRA, is used for this purpose. Both of these actions require the contents of the program counter to be sent to the register file. For this reason, multiplexer MuxY has a third input through which the return address can be routed to register RY, from where it can be sent to the register file. The return address is produced by the instruction address generator, as we will explain later.

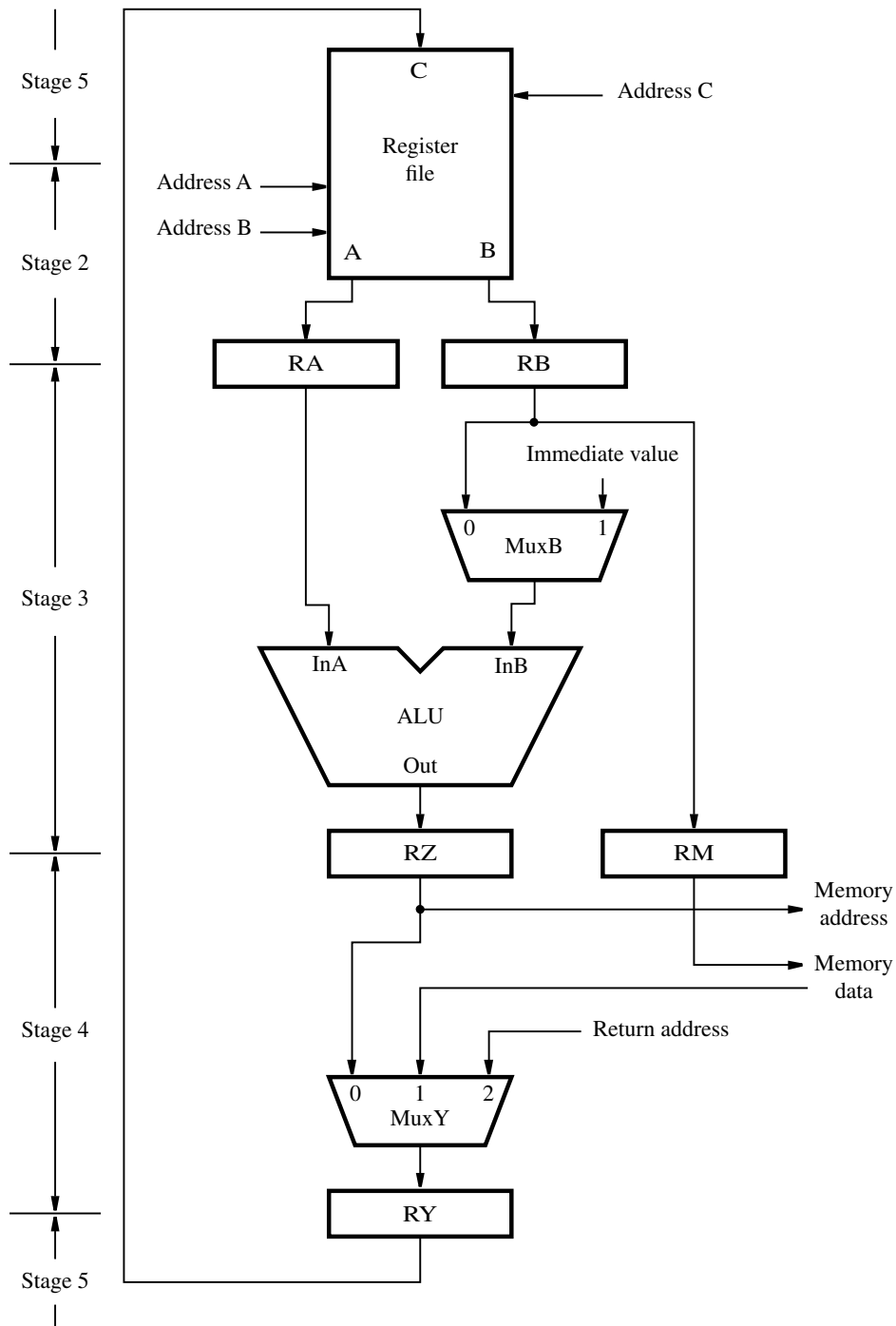


Figure 5.8 Datapath in a processor.

5.3.4 INSTRUCTION FETCH SECTION

The organization of the instruction fetch section of the processor is illustrated in Figure 5.9. The addresses used to access the memory come from the PC when fetching instructions and from register RZ in the datapath when accessing instruction operands. Multiplexer MuxMA selects one of these two sources to be sent to the processor-memory interface. The PC is included in a larger block, the instruction address generator, which updates the contents of the PC after each instruction is fetched. The instruction read from the memory is loaded into the IR, where it stays until its execution is completed and the next instruction is fetched.

The contents of the IR are examined by the control circuitry to generate the signals needed to control all the processor's hardware. They are also used by the block labeled Immediate. As described in Chapter 2, an immediate value may be included in some instructions. A 16-bit immediate value is extended to 32 bits. The extended value is then used either directly as an operand or to compute the effective address of an operand. For some instructions, such as those that perform arithmetic operations, the immediate value is sign-extended; for others, such as logic instructions, it is padded with zeros. The Immediate block in Figure 5.9 generates the extended value and forwards it to MuxB in Figure 5.8 to be used in an ALU computation. It also generates the extended value to be used in computing the target address of branch instructions.

The address generator circuit is shown in Figure 5.10. An adder is used to increment the PC by 4 during straight-line execution. It is also used to compute a new value to be

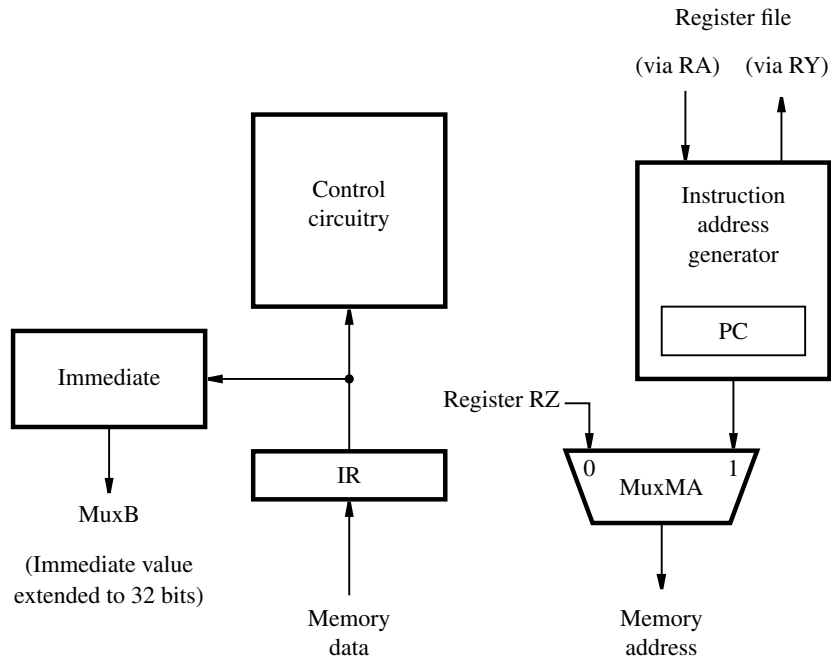


Figure 5.9 Instruction fetch section of Figure 5.7.

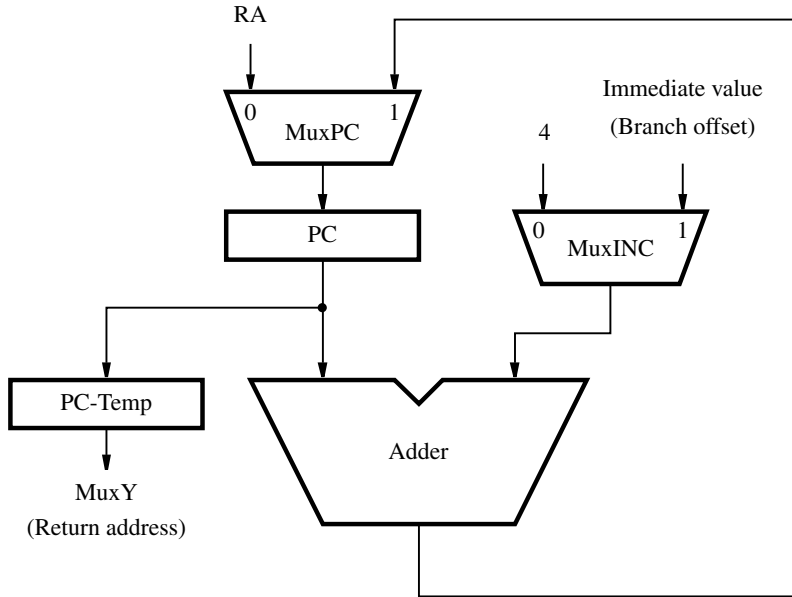


Figure 5.10 Instruction address generator.

loaded into the PC when executing branch and subroutine call instructions. One adder input is connected to the PC. The second input is connected to a multiplexer, MuxINC, which selects either the constant 4 or the branch offset to be added to the PC. The branch offset is given in the immediate field of the IR and is sign-extended to 32 bits by the Immediate block in Figure 5.9. The output of the adder is routed to the PC via a second multiplexer, MuxPC, which selects between the adder and the output of register RA. The latter connection is needed when executing subroutine linkage instructions. Register PC-Temp is needed to hold the contents of the PC temporarily during the process of saving the subroutine or interrupt return address.

5.4 INSTRUCTION FETCH AND EXECUTION STEPS

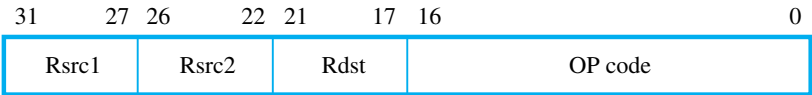
We now examine the process of fetching and executing instructions in more detail, using the datapath in Figure 5.8. Consider again the instruction

Add R3, R4, R5

The steps for fetching and executing this instruction are given in Figure 5.11. Assume that the instruction is encoded using the format in Figure 2.32, which is reproduced here as Figure 5.12. After the instruction has been fetched from the memory and placed in the IR, the source register addresses are available in fields IR_{31-27} and IR_{26-22} . These two fields

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, $IR \leftarrow$ Memory data, $PC \leftarrow$ [PC] + 4
2	Decode instruction, $RA \leftarrow$ [R4], $RB \leftarrow$ [R5]
3	$RZ \leftarrow$ [RA] + [RB]
4	$RY \leftarrow$ [RZ]
5	$R3 \leftarrow$ [RY]

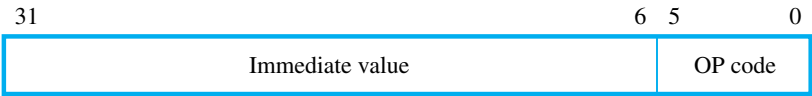
Figure 5.11 Sequence of actions needed to fetch and execute the instruction: Add R3, R4, R5.



(a) Register-operand format



(b) Immediate-operand format



(c) Call format

Figure 5.12 Instruction encoding.

are connected to the address inputs for ports A and B of the register file. As a result, registers R4 and R5 are read and their contents placed in registers RA and RB, respectively, at the end of step 2. In the next step, the control circuitry sets MuxB to select input 0, thus connecting register RB to input InB of the ALU. At the same time, it causes the ALU to perform an addition operation. Since register RA is connected to input InA, the ALU produces the required sum [RA] + [RB], which is loaded into register RZ at the end of step 3.

In step 4, multiplexer MuxY selects input 0, thus causing the contents of RZ to be transferred to RY. The control circuitry connects the destination address field of the Add instruction, IR_{21-17} , to the address input for port C of the register file. In step 5, it issues

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction, RA $\leftarrow$ [R7]
3	RZ $\leftarrow$ [RA] + Immediate value X
4	Memory address $\leftarrow$ [RZ], Read memory, RY $\leftarrow$ Memory data
5	R5 $\leftarrow$ [RY]

Figure 5.13 Sequence of actions needed to fetch and execute the instruction: Load R5, X(R7).

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction, RA $\leftarrow$ [R8], RB $\leftarrow$ [R6]
3	RZ $\leftarrow$ [RA] + Immediate value X, RM $\leftarrow$ [RB]
4	Memory address $\leftarrow$ [RZ], Memory data $\leftarrow$ [RM], Write memory
5	No action

Figure 5.14 Sequence of actions needed to fetch and execute the instruction: Store R6, X(R8).

a Write command to the register file, causing the contents of register RY to be written into register R3.

Load and Store instructions are executed in a similar manner. In this case, the address of the destination register is given in bit field IR_{26–22}. The control hardware connects this field to the address input corresponding to input C of the register file. The steps involved in executing these instructions are given in Figures 5.13 and 5.14. In both examples, the memory address is specified using the Index mode, in which the index value X is given as an immediate value in the instruction. The immediate field of IR, extended as appropriate by the Immediate block in Figure 5.9, is selected by MuxB in step 3 and added to the contents of register RA. The resulting sum is the effective address of the operand.

Some Observations

In the discussion above, we assumed that memory Read and Write operations can be completed in one clock cycle. Is this a realistic assumption? In general, accessing the main memory of a computer takes significantly longer than reading the contents of a register in the register file. However, most modern processors use cache memories, which will be discussed in detail in Chapter 8. A cache memory is much faster than the main memory.

It is usually implemented on the same chip as the processor, making it about as fast as the register file. Thus, a memory Read or Write operation can be completed in one clock cycle when the data involved are available in the cache. When the operation requires access to the main memory, the processor must wait for that operation to be completed. We will discuss how slower memory accesses are handled in Section 5.4.2.

We also assumed that the processor reads the source registers of the instruction in step 2, while it is still decoding the OP code of the instruction that has just been loaded into the IR. Can these two tasks be completed in the same step? How can the control hardware know which registers to read before it completes decoding the instruction? This is possible because source register addresses are specified using the same bit positions in all instructions. The hardware reads the registers whose addresses are in these bit positions once the instruction is loaded into the IR. Their contents are loaded into registers RA and RB at the end of step 2. If these data are needed by the instruction, they will be available for use in step 3. If not, they will be ignored by subsequent hardware stages.

Note that the actions described in Figures 5.11, 5.13, and 5.14 do not show two registers being read in step 2 in every case. To avoid confusion, only the registers needed by the specific instruction described in the figure are mentioned, even though two registers are always read.

5.4.1 BRANCHING

Instructions are fetched from sequential word locations in the memory during straight-line program execution. Whenever an instruction is fetched, the processor increments the PC by 4 to point to the next word. This execution pattern continues until a branch or subroutine call instruction loads a new address into the PC. Subroutine call instructions also save the return address, to be used when returning to the calling program. In this section we examine the actions needed to implement these instructions. Interrupts from I/O devices and software interrupt instructions are handled in a similar manner.

Branch instructions specify the branch target address relative to the PC. A branch offset given as an immediate value in the instruction is added to the current contents of the PC. The number of bits used for this offset is considerably less than the word length of the computer, because space is needed within the instruction to specify the OP code and the branch condition. Hence, the range of addresses that can be reached by a branch instruction is limited.

Subroutine call instructions can reach a larger range of addresses. Because they do not include a condition, more bits are available to specify the target address. Also, most RISC-style computers have Jump and Call instructions that use a general-purpose register to specify a full 32-bit address. The details vary from one computer to another, as the example processors introduced in Appendices B to E illustrate.

Branch Instructions

The sequence of steps for implementing an unconditional branch instruction is given in Figure 5.15. The instruction is fetched and the PC is incremented as usual in step 1. After the instruction has been decoded in step 2, multiplexer MuxINC selects the branch offset in

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction
3	PC $\leftarrow$ [PC] + Branch offset
4	No action
5	No action

Figure 5.15 Sequence of actions needed to fetch and execute an unconditional branch instruction.

the IR to be added to the PC in step 3. This is the address that will be used to fetch the next instruction. Execution of a Branch instruction is completed in step 3. No action is taken in steps 4 and 5.

We explained in Section 2.13 that the branch offset is the distance between the branch target and the memory location following the branch instruction. The reason for this can be seen clearly in Figure 5.15. The PC is incremented by 4 in step 1, at the time the branch instruction is fetched. Then, the branch target address is computed in step 3 by adding the branch offset to the updated contents of the PC.

The sequence in Figure 5.15 can be readily modified to implement conditional branch instructions. In processors that do not use condition-code flags, the branch instruction specifies a compare-and-test operation that determines the branch condition. For example, the instruction

Branch_if_[R5]=[R6] LOOP

results in a branch if the contents of registers R5 and R6 are identical. When this instruction is executed, the register contents are compared, and if they are equal, a branch is made to location LOOP.

Figure 5.16 shows how this instruction may be executed. Registers R5 and R6 are read in step 2, as usual, and compared in step 3. The comparison could be done by performing the subtraction operation $[R5] - [R6]$ in the ALU. The ALU generates signals that indicate whether the result of the subtraction is positive, negative, or zero. The ALU may also generate signals to show whether arithmetic overflow has occurred and whether the operation produced a carry-out. The control circuitry examines these signals to test the condition given in the branch instruction. In the example above, it checks whether the result of the subtraction is equal to zero. If it is, the branch target address is loaded into the PC, to be used to fetch the next instruction. Otherwise, the contents of the PC remain at the incremented value computed in step 1, and straight-line execution continues.

According to the sequence of steps in Figure 5.16, the two actions of comparing the register contents and testing the result are both carried out in step 3. Hence, the clock cycle must be long enough for the two actions to be completed, one after the other. For this reason, it is desirable that the comparison be done as quickly as possible. A subtraction

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction, RA $\leftarrow$ [R5], RB $\leftarrow$ [R6]
3	Compare [RA] to [RB], If [RA] = [RB], then PC $\leftarrow$ [PC] + Branch offset
4	No action
5	No action

Figure 5.16 Sequence of actions needed to fetch and execute the instruction: Branch_if_[R5]=[R6] LOOP.

operation in the ALU is time consuming, and is not needed in this case. A simpler and faster comparator circuit can examine the contents of registers RA and RB and produce the required condition signals, which indicate the conditions greater than, equal, less than, etc. A comparator is not shown separately in Figure 5.8 as it can be a part of the ALU block. Example 5.3 shows how a comparator circuit can be designed.

Subroutine Call Instructions

Subroutine calls and returns are implemented in a similar manner to branch instructions. The address of the subroutine may either be computed using an immediate value given in the instruction or it may be given in full in one of the general-purpose registers. Figure 5.17 gives the sequence of actions for the instruction

Call_Register R9

which calls a subroutine whose address is in register R9. The contents of that register are read and placed in RA in step 2. During step 3, multiplexer MuxPC selects its 0 input, thus transferring the data in register RA to be loaded into the PC.

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction, RA $\leftarrow$ [R9]
3	PC-Temp $\leftarrow$ [PC], PC $\leftarrow$ [RA]
4	RY $\leftarrow$ [PC-Temp]
5	Register LINK $\leftarrow$ [RY]

Figure 5.17 Sequence of actions needed to fetch and execute the instruction: Call_Register R9.

Assume that the return address of the subroutine, which is the previous contents of the PC, is to be saved in a general-purpose register called LINK in the register file. Data are written into the register file in step 5. Hence, it is not possible to send the return address directly to the register file in step 3. To maintain correct data flow in the five-stage structure, the processor saves the return address in a temporary register, PC-Temp. From there, the return address is transferred to register RY in step 4, then to register LINK in step 5. The address LINK is built into the control circuitry.

Subroutine return instructions transfer the value saved in register LINK back to the PC. The encoding of the Return-from-subroutine instruction is such that the address of register LINK appears in bits IR_{31–27}. This is the field connected to Address A of the register file. Hence, once the instruction is fetched, register LINK is read and its contents are placed in RA, from where they can be transferred to the PC via MuxPC in Figure 5.10. Return-from-interrupt instructions are handled in a similar manner, except that a different register is used to hold the return address.

5.4.2 WAITING FOR MEMORY

The role of the processor-memory interface circuit is to control data transfers between the processor and the memory. We pointed out earlier that modern processors use fast, on-chip cache memories. Most of the time, the instruction or data referenced in memory Read and Write operations are found in the cache, in which case the operation is completed in one clock cycle. When the requested information is not in the cache and has to be fetched from the main memory, several clock cycles may be needed. The interface circuit must inform the processor's control circuitry about such situations, to delay subsequent execution steps until the memory operation is completed.

Assume that the processor-memory interface circuit generates a signal called Memory Function Completed (MFC). It asserts this signal when a requested memory Read or Write operation has been completed. The processor's control circuitry checks this signal during any processing step in which it issues a memory Read or Write request, to determine when it can proceed to the next step. When the requested data are found in the cache, the interface circuit asserts the MFC signal before the end of the same clock cycle in which the memory request is issued. Hence, instruction execution continues uninterrupted. If access to the main memory is required, the interface circuit delays asserting MFC until the operation is completed. In this case, the processor's control circuitry must extend the duration of the execution step for as many clock cycles as needed, until MFC is asserted. We will use the command Wait for MFC to indicate that a given execution step must be extended, if necessary, until a memory operation is completed. When MFC is received, the actions specified in the step are completed, and the processor proceeds to the next step in the execution sequence.

Step 1 of the execution sequence of any instruction involves fetching the instruction from the memory. Therefore, it must include a Wait for MFC command, as follows:

Memory address $\leftarrow$ [PC], Read memory, Wait for MFC,
IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4

The Wait for MFC command is also needed in step 4 of Load and Store instructions in Figures 5.13 and 5.14. Most of the time, the requested information is found in the cache, so the MFC signal is generated quickly, and the step is completed in one clock cycle. When an access involves the main memory, the MFC response is delayed, and the step is extended to several clock cycles.

5.5 CONTROL SIGNALS

The operation of the processor's hardware components is governed by *control signals*. These signals determine which multiplexer input is selected, what operation is performed by the ALU, and so on. In this section we discuss the signals needed to control the operation of the components in Figures 5.8 to 5.10.

It is instructive to begin by recalling how data flow through the four stages of the datapath, as described in Section 5.3.3. In each clock cycle, the results of the actions that take place in one stage are stored in inter-stage registers, to be available for use by the next stage in the next clock cycle. Since data are transferred from one stage to the next in every clock cycle, inter-stage registers are always enabled. This is the case for registers RA, RB, RZ, RY, RM, and PC-Temp. The contents of the other registers, namely, the PC, the IR, and the register file, must not be changed in every clock cycle. New data are loaded into these registers only when called for in a particular processing step. They must be enabled only at those times.

The role of the multiplexers is to select the data to be operated on in any given stage. For example, MuxB in stage 3 of Figure 5.8 selects the immediate field in the IR for instructions that use an immediate source operand. It also selects that field for instructions that use immediate data as an offset when computing the effective address of a memory operand. Otherwise, it selects register RB. The data selected by the multiplexer are used by the ALU. Examination of Figures 5.11, 5.13, and 5.14 shows that the ALU is used only in step 3, and hence the selection made by MuxB matters only during that step. To simplify the required control circuit, the same selection can be maintained in all execution steps. A similar observation can be made about MuxY. However, MuxMA in Figure 5.9 must change its selection in different execution steps. It selects the PC as the source of the memory address during step 1, when a new instruction is being fetched. During step 4 of Load and Store instructions, it selects register RZ, which contains the effective address of the memory operand.

Figures 5.18, 5.19, and 5.20 show the required control signals. The register file has three 5-bit address inputs, allowing access to 32 general-purpose registers. Two of these inputs, Address A and Address B, determine which registers are to be read. They are connected to fields IR_{31–27} and IR_{26–22} in the instruction register. The third address input, Address C, selects the destination register, into which the input data at port C are to be written. Multiplexer MuxC selects the source of that address. We have assumed that three-register instructions use bits IR_{21–17} and other instructions use IR_{26–22} to specify the destination register, as in Figure 5.12. The third input of the multiplexer is the address of the link register used in subroutine linkage instructions. New data are loaded into the selected register only when the control signal RF_write is asserted.

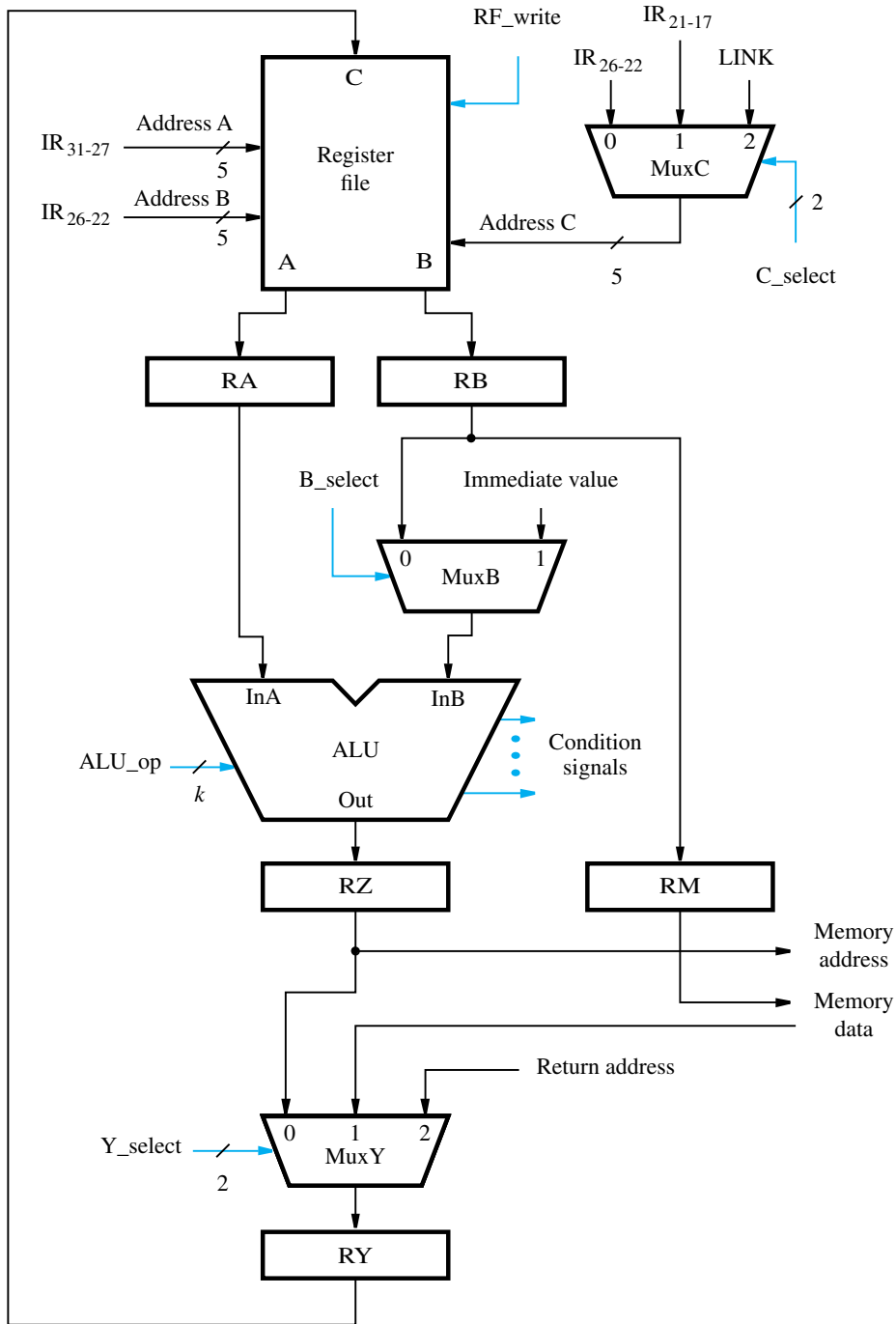


Figure 5.18 Control signals for the datapath.

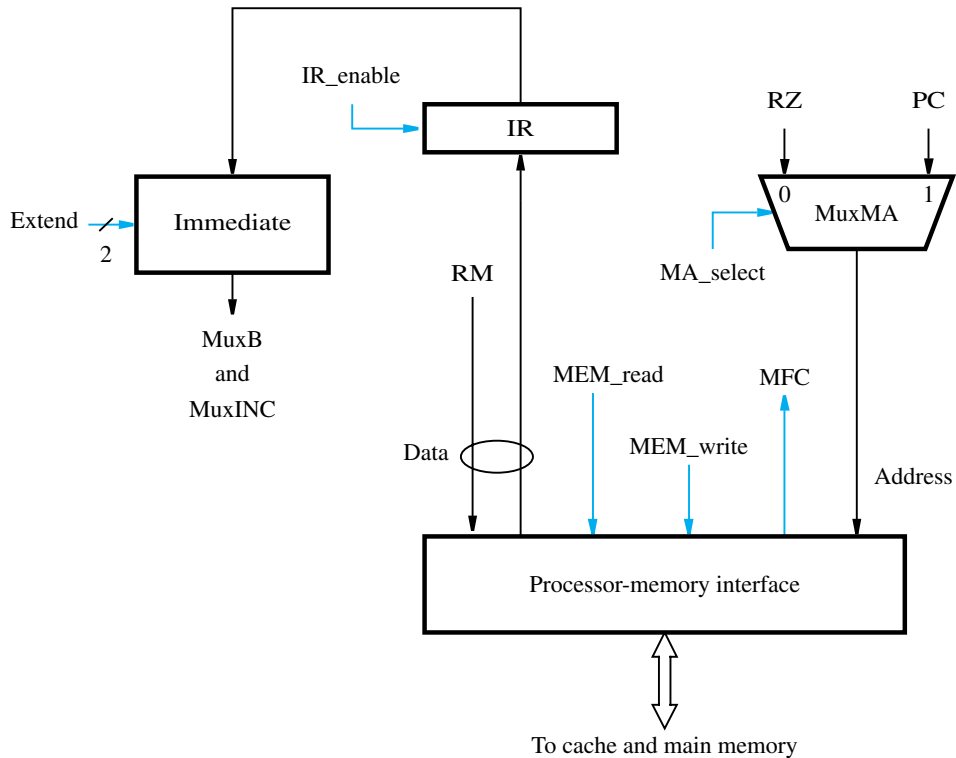


Figure 5.19 Processor-memory interface and IR control signals.

Multiplexers are controlled by signals that select which input data appear at the multiplexer's output. For example, when B_select is equal to 0, MuxB selects the contents of register RB to be available at input InB of the ALU. Note that two bits are needed to control MuxC and MuxY, because each multiplexer selects one of three inputs.

The operation performed by the ALU is determined by a k -bit control code, ALU_op , which can specify up to 2^k distinct operations, such as Add, Subtract, AND, OR, and XOR. When an instruction calls for two values to be compared, a comparator performs the comparison specified, as mentioned earlier. The comparator generates condition signals that indicate the result of the comparison. These signals are examined by the control circuitry during the execution of conditional branch instructions to determine whether the branch condition is true or false.

The interface between the processor and the memory and the control signals associated with the instruction register are presented in Figure 5.19. Two signals, MEM_read and MEM_write are used to initiate a memory Read or a memory Write operation. When the requested operation has been completed, the interface asserts the MFC signal. The instruction register has a control signal, IR_enable , which enables a new instruction to be loaded into the register. During a fetch step, it must be activated only after the MFC signal is asserted.

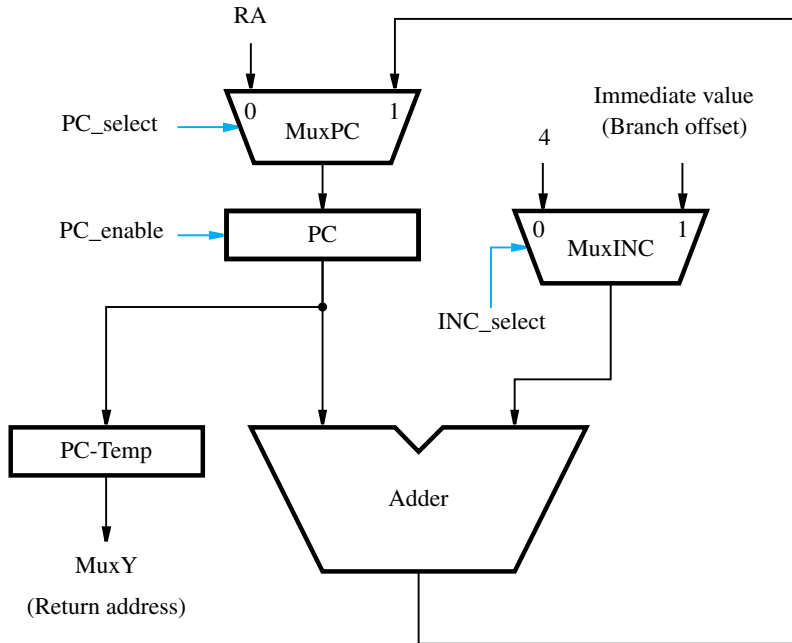


Figure 5.20 Control signals for the instruction address generator.

We have assumed that the Immediate block handles three possible formats for the immediate value: a sign-extended 16-bit value, a zero-extended 16-bit value, and a 26-bit value that is handled in a special way (see Problem 5.14). Hence, its control signal, *Extend*, comprises two bits.

The signals that control the operation of the instruction address generator are shown in Figure 5.20. The *INC_select* signal selects the value to be added to the PC, either the constant 4 or the branch offset specified in the instruction. The *PC_select* signal selects either the updated address or the contents of register RA to be loaded into the PC when the *PC_enable* control signal is activated.

5.6 HARDWIRED CONTROL

Previous sections described the actions needed to fetch and execute instructions. We now examine how the processor generates the control signals that cause these actions to take place in the correct sequence and at the right time. There are two basic approaches: hardwired control and microprogrammed control. Hardwired control is discussed in this section.

An instruction is executed in a sequence of steps, where each step requires one clock cycle. Hence, a step counter may be used to keep track of the progress of execution. Several

actions are performed in each step, depending on the instruction being executed. In some cases, such as for branch instructions, the actions taken depend on tests applied to the result of a computation or a comparison operation. External signals, such as interrupt requests, may also influence the actions to be performed. Thus, the setting of the control signals depends on:

- Contents of the step counter
- Contents of the instruction register
- The result of a computation or a comparison operation
- External input signals, such as interrupt requests

The circuitry that generates the control signals may be organized as shown in Figure 5.21. The instruction decoder interprets the OP-code and addressing mode information in the IR and sets to 1 the corresponding INS_i output. During each clock cycle, one of the outputs T1 to T5 of the step counter is set to 1 to indicate which of the five steps involved in fetching and executing instructions is being carried out. Since all instructions are completed in five steps, a modulo-5 counter may be used. The control signal generator is a combinational circuit that produces the necessary control signals based on all its inputs. The required settings of the control signals can be determined from the action sequences that implement each of the instructions represented by the signals INS_1 to INS_m .

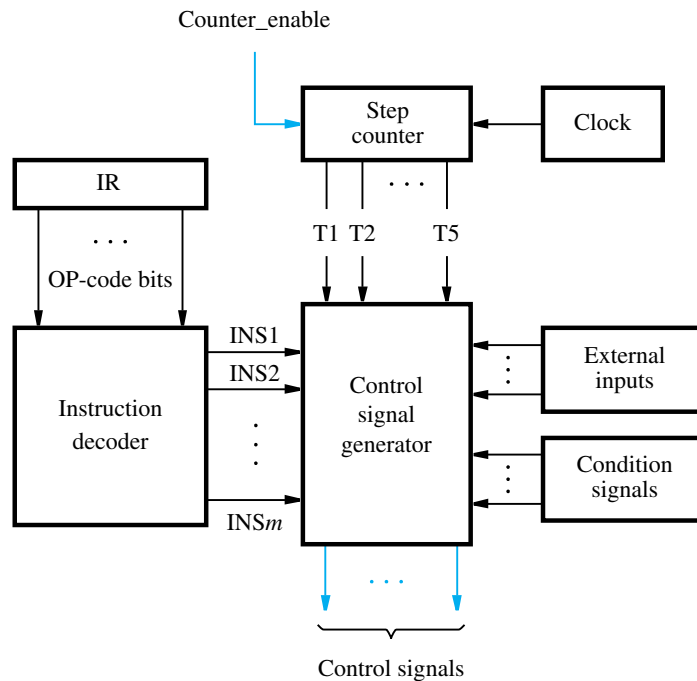


Figure 5.21 Generation of the control signals.

As an example, consider step 1 in the instruction execution process. This is the step in which a new instruction is fetched from the memory. It is identified by signal T1 being asserted. During that clock period, the MA_select signal in Figure 5.19 is set to 1 to select the PC as the source of the memory address, and MEM_read is activated to initiate a memory Read operation. The data received from the memory are loaded into the IR by activating IR_enable when the memory's response signal, MFC, is asserted. At the same time, the PC is incremented by 4, by setting the INC_select signal in Figure 5.20 to 0 and PC_select to 1. The PC_enable signal is activated to cause the new value to be loaded into the PC at the positive edge of the clock marking the end of step T1.

5.6.1 DATAPATH CONTROL SIGNALS

Instructions that handle data include Load, Store, and all computational instructions. They perform various data movement and manipulation operations using the processor's datapath, whose control signals are shown in Figures 5.18 and 5.19. Once an instruction is loaded into the IR, the instruction decoder interprets its contents to determine the actions needed. At the same time, the source registers are read and their contents become available at the A and B outputs of the register file. As mentioned earlier, inter-stage registers RA, RB, RZ, RM, and RY are always enabled. This means that data flow automatically from one datapath stage to the next on every active edge of the clock signal.

The desired setting of various control signals can be determined by examining the actions taken in each execution step of every instruction. For example, the RF_write signal is set to 1 in step T5 during execution of an instruction that writes data into the register file. It may be generated by the logic expression

$$\text{RF_write} = \text{T5} \cdot (\text{ALU} + \text{Load} + \text{Call})$$

where ALU stands for all instructions that perform arithmetic or logic operations, Load stands for all Load instructions, and Call stands for all subroutine-call and software-interrupt instructions. The RF_write signal is a function of both the instruction and the timing signals. But, as mentioned earlier, the setting of some of the multiplexers need not change from one timing step to another. In this case, the multiplexer's select signal can be implemented as a function of the instruction only. For example,

$$\text{B_select} = \text{Immediate}$$

where Immediate stands for all instructions that use an immediate value in the IR. We encourage the reader to examine other control signals and derive the appropriate logic expressions for them, based on the execution steps of various instructions.

5.6.2 DEALING WITH MEMORY DELAY

The timing signals T1 to T5 are asserted in sequence as the step counter is advanced. Most of the time, the step counter is incremented at the end of every clock cycle. However, a step

in which a MEM_read or a MEM_write command is issued does not end until the MFC signal is asserted, indicating that the requested memory operation has been completed.

To extend the duration of an execution step to more than one clock cycle, we need to disable the step counter. Assume that the counter is incremented when enabled by a control signal called Counter_enable. Let the need to wait for a memory operation to be completed be indicated by a control signal called WMFC, which is activated during any execution step in which the Wait for MFC command is issued. Counter_enable should be set to 1 in any step in which WMFC is not asserted. Otherwise, it should be set to 0 when MFC is asserted. This means that

$$\text{Counter_enable} = \overline{\text{WMFC}} + \text{MFC}$$

A new value is loaded into the PC at the end of any clock cycle in which the PC_enable signal in Figure 5.20 is activated. We must ensure that the PC is incremented only once when an execution step is extended for more than one clock cycle. Hence, when fetching an instruction, the PC should be enabled only when MFC is received. It is also enabled in step 3 of instructions that cause branching. Let BR denote all instructions in this group. Then, PC_enable may be realized as

$$\text{PC_enable} = T1 \cdot \text{MFC} + T3 \cdot \text{BR}$$

5.7 CISC-STYLE PROCESSORS

We saw in the previous sections that a RISC-style instruction set is conducive to a multi-stage implementation of the processor. All instructions can be executed in a uniform manner using the same five-stage hardware. As a result, the hardware is simple and well suited to pipelined operation. Also, the control signals are easy to generate.

CISC-style instruction sets are more complex because they allow much greater flexibility in accessing instruction operands. Unlike RISC-style instruction sets, where only Load and Store instructions access data in the memory, CISC instructions can operate directly on memory operands. Also, they are not restricted to one word in length. An instruction may use several words to specify operand addresses and the actions to be performed, as explained in Section 2.10. Therefore, CISC-style instructions require a different organization of the processor hardware.

Figure 5.22 shows a possible processor organization. The main difference between this organization and the five-stage structure discussed earlier is that the Interconnect block, which provides interconnections among other blocks, does not prescribe any particular structure or pattern of data flow. It provides paths that make it possible to transfer data between any two components, as needed to implement instructions. The multi-stage structure of Figure 5.8 uses inter-stage registers, such as RZ and RY. These are not needed in the organization of Figure 5.22. Instead, some registers are needed to hold intermediate results during instruction execution. The temporary registers block in the figure is provided for this purpose. It includes two temporary registers, Temp1 and Temp2. The need for these registers will become apparent from the examples given later.

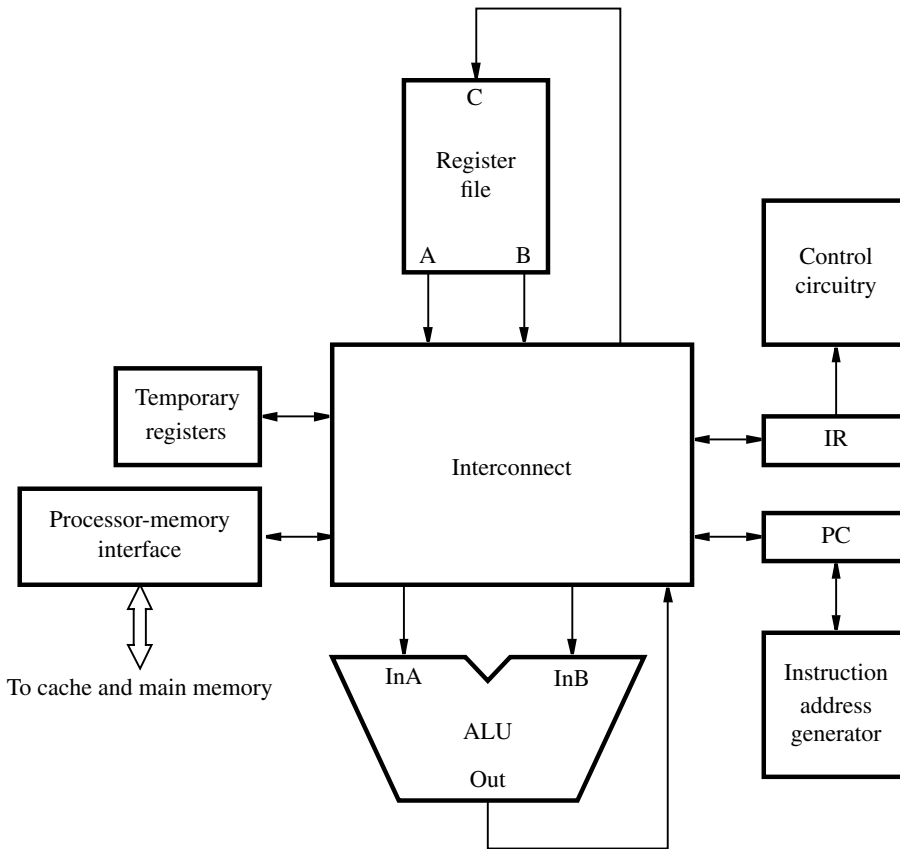


Figure 5.22 Organization of a CISC-style processor.

A traditional approach to the implementation of the Interconnect is to use buses. A *bus* consists of a set of lines to which several devices may be connected, enabling data to be transferred from any one device to any other. A logic gate that sends a signal over a bus line is called a *bus driver*. Since all devices connected to the bus have the ability to send data, we must ensure that only one of them is driving the bus at any given time. For this reason, the bus driver is a special type of logic gate called a *tri-state gate*. It has a control input that turns it on or off. When turned on, the gate places a logic signal of 0 or 1 on the bus, according to the value of its input. When turned off, the gate is electrically disconnected from the bus, as explained in Appendix A.

Figure 5.23 shows how a flip-flop that forms one bit of a data register can be connected to a bus line. There are two control signals, R_{in} and R_{out} . When R_{in} is equal to 1 the multiplexer selects the data on the bus line to be loaded into the flip-flop. Setting R_{in} to 0 causes the flip-flop to maintain its present value. The output of the flip-flop is connected to the bus line through a tri-state gate, which is turned on when R_{out} is asserted. At other times, the tri-state gate is turned off, allowing other components to drive the bus line.

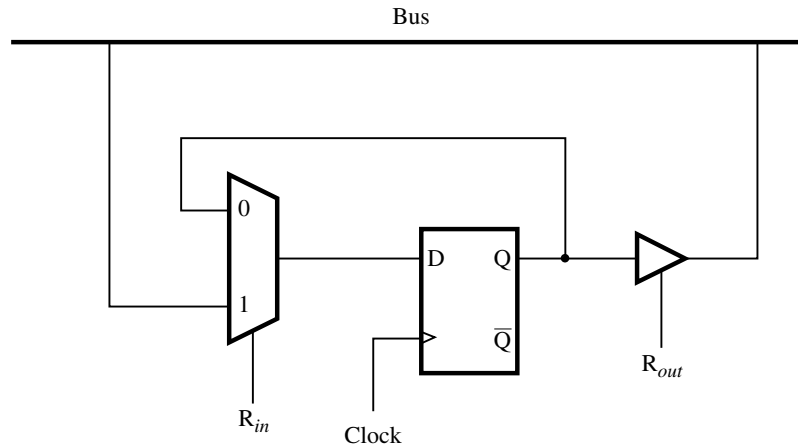


Figure 5.23 Input and output gating for one register bit.

5.7.1 AN INTERCONNECT USING BUSES

The Interconnect in Figure 5.22 may be implemented using one or more buses. Figure 5.24 shows a three-bus implementation. All registers are assumed to be edge-triggered. That is, when a register is enabled, data are loaded into it on the active edge of the clock at the end of the clock period. Addresses for the three ports of the register file are provided by the Control block. These connections are not shown to keep the figure simple. Also not shown is the Immediate block through which the IR is connected to bus B. This is the circuit that extends an immediate operand in the IR to 32 bits.

Consider the two-operand instruction

Add R5, R6

which performs the operation

$$R5 \leftarrow [R5] + [R6]$$

Fetching and executing this instruction using the hardware in Figure 5.24 can be performed in three steps, as shown in Figure 5.25. Each step, except for the step involving access to the memory, is completed in one clock cycle. In step 1, bus B is used to send the contents of the PC to the processor-memory interface, which sends them on the memory address lines and initiates a memory Read operation. The data received from the memory, which represent an instruction to be executed, are sent to the IR over bus C. The command Wait for MFC is included to accommodate the possibility that memory access may take more than one clock cycle, as explained in Section 5.4.2. The instruction is decoded in step 2 and the control circuitry begins reading the source registers, R5 and R6. However, the contents of the registers do not become available at the A and B outputs of the register file until step 3. They are sent to the ALU using buses A and B. The ALU performs the addition operation, and the sum is sent back to the ALU over bus C, to be written into register R5 at the end of the clock cycle.

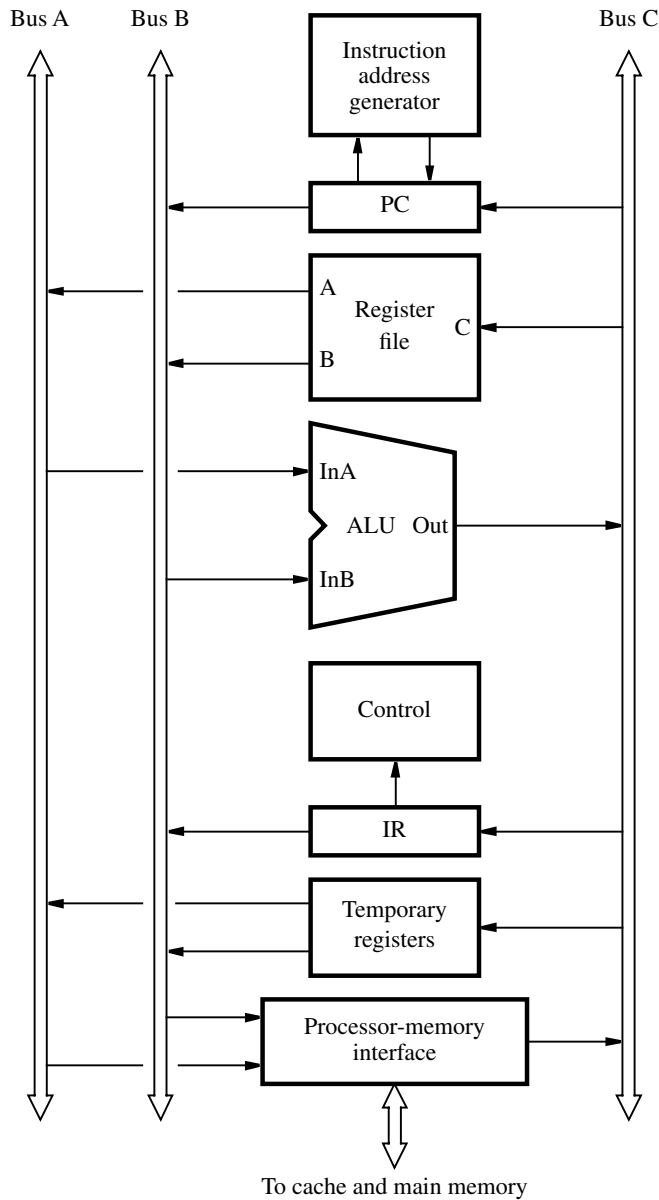


Figure 5.24 Three-bus CISC-style processor organization.

Note that reading the source registers is completed in step 2 in Figure 5.11. In that case, the action of reading the registers proceeds in parallel with the action of decoding the instruction, because the location of the bit fields containing register addresses in a RISC-style instruction is known. Since CISC-style instructions do not always use the same

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, Wait for MFC, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction
3	R5 $\leftarrow$ [R5] + [R6]

Figure 5.25 Sequence of actions needed to fetch and execute the instruction: Add R5, R6.

instruction fields to specify register addresses, the action of reading the source registers does not begin until the instruction has been at least partially decoded. Hence, it may not be possible to complete reading the source registers in step 2.

Next, consider the instruction

And X(R7), R9

which performs the logical AND operation on the contents of register R9 and memory location $X + [R7]$ and stores the result back in the same memory location. Assume that the index offset X is a 32-bit value given as the second word of the instruction. To execute this instruction, it is necessary to access the memory four times. First, the OP-code word is fetched. Then, when the instruction decoding circuit recognizes the Index addressing mode, the index offset X is fetched. Next, the memory operand is fetched and the AND operation is performed. Finally, the result is stored back into the memory.

Figure 5.26 gives the steps needed to execute the instruction. After decoding the instruction in step 2, the second word of the instruction is read in step 3. The data received,

Step	Action
1	Memory address $\leftarrow$ [PC], Read memory, Wait for MFC, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2	Decode instruction
3	Memory address $\leftarrow$ [PC], Read memory, Wait for MFC, Temp1 $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
4	Temp2 $\leftarrow$ [Temp1] + [R7]
5	Memory address $\leftarrow$ [Temp2], Read memory, Wait for MFC, Temp1 $\leftarrow$ Memory data
6	Temp1 $\leftarrow$ [Temp1] AND [R9]
7	Memory address $\leftarrow$ [Temp2], Memory data $\leftarrow$ [Temp1], Write memory, Wait for MFC

Figure 5.26 Sequence of actions needed to fetch and execute the instruction: And X(R7), R9.

which represent the offset X , are stored temporarily in register Temp1, to be used in the next step for computing the effective address of the memory operand. In step 4, the contents of registers Temp1 and R7 are sent to the ALU inputs over buses A and B. The effective address is computed and placed into register Temp2, then used to read the operand in step 5. Register Temp1 is used again during step 5, this time to hold the data operand received from the memory. The computation is performed in step 6, and the result is placed back in register Temp1. In the final step, the result is sent to be stored in the memory at the operand address, which is still available in register Temp2.

The two examples in Figures 5.25 and 5.26 illustrate the variability in the number of execution steps in CISC-style instructions. There is no uniform sequence of actions that can be followed for all instructions in the same way as was demonstrated for RISC instructions in Section 5.2.

5.7.2 MICROPROGRAMMED CONTROL

The control signals needed to control the operation of the components in Figures 5.22 and 5.24 can be generated using the hardwired approach described in Section 5.6. But, there is an interesting alternative that was popular in the past, which we describe next.

Control signals are generated for each execution step based on the instruction in the IR. In hardwired control, these signals are generated by circuits that interpret the contents of the IR as well as the timing signals derived from a step counter. Instead of employing such circuits, it is possible to use a “software” approach, in which the desired setting of the control signals in each step is determined by a program stored in a special memory. The control program is called a *microprogram* to distinguish it from the program being executed by the processor. The microprogram is stored on the processor chip in a small and fast memory called the *microprogram memory* or the *control store*.

Suppose that n control signals are needed. Let each control signal be represented by a bit in an n -bit word, which is often referred to as a *control word* or a *microinstruction*. Each bit in that word specifies the setting of the corresponding signal for a particular step in the execution flow. One control word is stored in the microprogram memory for each step in the execution sequence of an instruction. For example, the action of reading an instruction or a data operand from the memory requires use of the MEM_read and WMFC signals introduced in Sections 5.5 and 5.6.2, respectively. These signals are asserted by setting the corresponding bits in the control word to 1 for steps 1, 3, and 5 in Figure 5.26. When a microinstruction is read from the control store, each control signal takes on the value of its corresponding bit.

The sequence of microinstructions corresponding to a given machine instruction constitutes the *microroutine* that implements that instruction. The first two steps in Figures 5.25 and 5.26 specify the actions for fetching and decoding an instruction. They are common to all instructions. The microroutine that is specific to a given machine instruction starts with step 3.

Figure 5.27 depicts a typical organization of the hardware needed for microprogrammed control. It consists of a microinstruction address generator, which generates the address

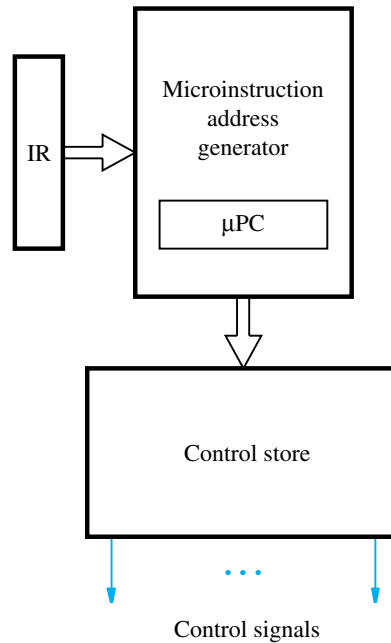


Figure 5.27 Microprogrammed control unit organization.

to be used for reading microinstructions from the control store. The address generator uses a *microprogram counter*, μ PC, to keep track of control store addresses when reading microinstructions from successive locations. During step 2 in Figures 5.25 and 5.26, the microinstruction address generator decodes the instruction in the IR to obtain the starting address of the corresponding microroutine and loads that address into the μ PC. This is the address that will be used in the following clock cycle to read the control word corresponding to step 3. As execution proceeds, the microinstruction address generator increments the μ PC to read microinstructions from successive locations in the control store. One bit in the microinstruction, which we will call *End*, is used to mark the last microinstruction in a given microroutine. When *End* is equal to 1, as would be the case in step 3 in Figure 5.25 and step 7 in Figure 5.26, the address generator returns to the microinstruction corresponding to step 1, which causes a new machine instruction to be fetched.

Microprogrammed control can be viewed as having a control processor within the main processor. Microinstructions are fetched and executed much like machine instructions. Their function is to direct the actions of the main processor's hardware components, by indicating which control signals need to be active during each execution step.

Microprogrammed control is simple to implement and provides considerable flexibility in controlling the execution of machine instructions. But, it is slower than hardwired control. Also, the flexibility it provides is not needed in RISC-style processors. As the discussion in this chapter illustrates, the control signals needed to implement RISC-style instructions are

quite simple to generate. Since the cost of logic circuitry is no longer a significant factor, hardwired control has become the preferred choice.

5.8 CONCLUDING REMARKS

This chapter explained the basic structure of a processor and how it executes instructions. Modern processors have a multi-stage organization because this is a structure that is well-suited to pipelined operation. Each stage implements the actions needed in one of the execution steps of an instruction. A five-step sequence in which each step is completed in one clock cycle has been demonstrated. Such an approach is commonly used in processors that have a RISC-style instruction set.

The discussion in this chapter assumed that the execution of one instruction is completed before the next instruction is fetched. Only one of the five hardware stages is used at any given time, as execution moves from one stage to the next in each clock cycle. We will show in the next chapter that it is possible to overlap the execution steps of successive instructions, resulting in much better performance. This leads to a pipelined organization.

5.9 SOLVED PROBLEMS

This section presents some examples of the types of problems that a student may be asked to solve, and shows how such problems can be solved.

Problem: Figure 5.11 shows an Add instruction being executed in five steps, but no processing actions take place in step 4. If it is desired to eliminate that step, what changes have to be made in the datapath in Figure 5.8 to make this possible?

Example 5.1

Solution: Step 4 can be skipped by sending the output of the ALU in Figure 5.8 directly to register RY. This can be accomplished by adding one more input to multiplexer MuxY and connecting that input to the output of the ALU. Thus, the result of a computation at the output of the ALU is loaded into both registers RZ and RY at the end of step 3. For an Add instruction, or any other computational instruction, the register file control signal RF_write can be enabled in step 4 to load the contents of RY into the register file.

Problem: Assume that all memory access operations are completed in one clock cycle in a processor that has a 1-GHz clock. What is the frequency of memory access operations if Load and Store instructions constitute 20 percent of the dynamic instruction count in a program? (The dynamic count is the number of instruction executions, including the effect of program loops, which may cause some instructions to be executed more than once.) Assume that all instructions are executed in 5 clock cycles.

Example 5.2

Solution: There is one memory access to fetch each instruction. Then, 20 percent of the instructions have a second memory access to read or write a memory operand. On average, each instruction has 1.2 memory accesses in 5 clock cycles. Therefore, the frequency of memory accesses is $(1.2/5) \times 10^9$, or 240 million accesses per second.

Example 5.3 Problem: Derive the logic expressions for a circuit that compares two unsigned numbers: $X = x_2x_1x_0$ and $Y = y_2y_1y_0$ and generates three outputs: XGY , XEY , and XLY . One of these outputs is set to 1 to indicate that X is greater than, equal to, or less than Y , respectively.

Solution: To compare two unsigned numbers, we need to compare individual bit locations, starting with the most significant bit. If $x_2 = 1$ and $y_2 = 0$, then X is greater than Y . If $x_2 = y_2$, then we need to compare the next lower bit location, and so on. Thus, the logic expressions for the three outputs may be written as follows:

$$XGY = x_2\bar{y}_2 + \overline{(x_2 \oplus y_2)} \cdot (x_1\bar{y}_1 + \overline{(x_1 \oplus y_1)} x_0\bar{y}_0)$$

$$XEY = \overline{(x_2 \oplus y_2)} \cdot \overline{(x_1 \oplus y_1)} \cdot \overline{(x_0 \oplus y_0)}$$

$$XLY = \overline{XGY + XEY}$$

Example 5.4 Problem: Give the sequence of actions for a Return-from-subroutine instruction in a RISC processor. Assume that the address LINK of the general-purpose register in which the subroutine return address is stored is given in the instruction field connected to address A of the register file (IR_{31–27}).

Solution: Whenever an instruction is loaded into the IR, the contents of the general-purpose register whose address is given in bits IR_{31–27} are read and placed into register RA (see Figure 5.18). Hence, a Return-from-subroutine instruction will cause the contents of register LINK to be read and placed in register RA. Execution proceeds as follows:

1. Memory address $\leftarrow$ [PC], Read memory, Wait for MFC, IR $\leftarrow$ Memory data, PC $\leftarrow$ [PC] + 4
2. Decode instruction, RA $\leftarrow$ [LINK]
3. PC $\leftarrow$ [RA]
4. No action
5. No action

Example 5.5 Problem: A processor has the following interrupt structure. When an interrupt is received, the interrupt return address is saved in a general-purpose register called IRA. The current contents of the processor status register, PS, are saved in a special register called IPS, which is not a general-purpose register. The interrupt-service routine starts at address ILOC.

Assume that the processor checks for interrupts in the last execution step of every instruction. If an interrupt request is present and interrupts are enabled, the request is accepted. Instead of fetching the next instruction, the processor saves the PC and the PS and branches to ILOC. Give a suitable sequence of steps for performing these actions. What additional hardware is needed in Figures 5.18 to 5.20 to support interrupt processing?

Solution: The first two steps of instruction execution, in which an instruction is fetched and decoded, are not needed in the case of an interrupt. They may be skipped, or they would take no action if it is desired to maintain a 5-step sequence. Saving the PC can be done in exactly the same manner as for a subroutine call instruction. Another input to MuxC in Figure 5.18 is needed to which the address of register IRA should be connected. To load the starting address of the interrupt-service routine into the PC, an additional input to MuxPC in Figure 5.20 is needed, to which the value ILOC should be connected. Registers PS and IPS should be connected directly to each other to enable data to be transferred between them. The execution steps required are:

3. $PC\text{-Temp} \leftarrow [PC]$, $PC \leftarrow ILOC$, $IPS \leftarrow [PS]$, Disable interrupts
4. $RY \leftarrow [PC\text{-Temp}]$
5. $IRA \leftarrow [RY]$

These actions are reversed by a Return-from-interrupt instruction. See Problem 5.8.

Problem: Example 5.5 illustrates how the contents of the PC and the PS are saved when an interrupt request is accepted. In order to support interrupt nesting, it is necessary for the interrupt-service routine to save these registers on the processor stack, as described in Section 3.2. To do so, the contents of the PS, which are saved in register IPS at the time the interrupt is accepted, need to be moved to one of the general-purpose registers, from where they can be saved on the stack. Assume that two special instructions

Example 5.6

MoveControl R_i , IPS

and

MoveControl IPS, R_i

are available to save and restore the contents of IPS, respectively. Suggest changes to the hardware in Figures 5.8 and 5.10 to implement these instructions.

Solution: A possible organization is shown in Figure 5.28. To save the contents of IPS, its output is connected to an additional input on MuxY. When restoring its contents, MuxIPS selects register RA.

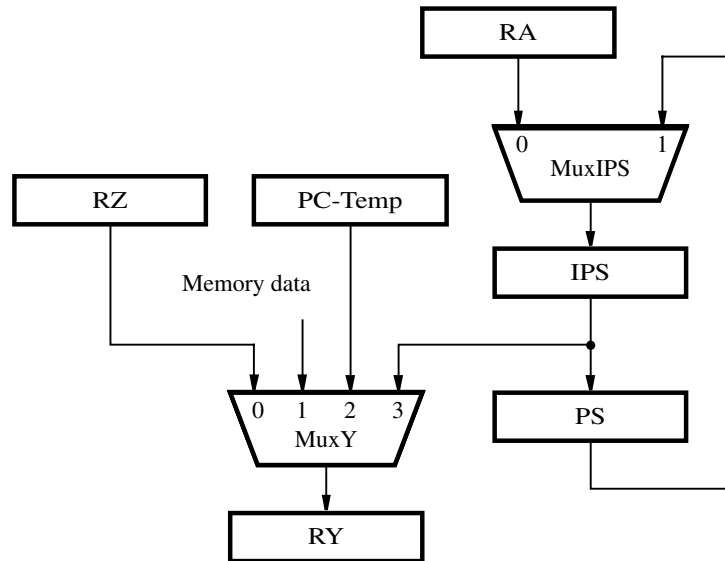


Figure 5.28 Connection of IPS for Example 5.6.

PROBLEMS

- 5.1** [M] The propagation delay through the combinational circuit in Figure 5.2 is 600 ps (picoseconds). The registers have a setup time requirement of 50 ps, and the maximum propagation delay from the clock input to the Q outputs is 70 ps.

(a) What is the minimum clock period required for correct operation of this circuit?

(b) Assume that the circuit is reorganized into three stages as in Figure 5.3, such that the combinational circuit in each stage has a delay of 200 ps. What is the minimum clock period in this case?

- 5.2** [M] At the time the instruction

Load R6, 1000(R9)

is fetched, R6 and R9 contain the values 4200 and 85320, respectively. Memory location 86320 contains 75900. Show the contents of the interstage registers in Figure 5.8 during each of the 5 execution steps of this instruction.

- 5.3** [E] Figure 5.12 shows the bit fields assigned to register addresses for different groups of instructions. Why is it important to use the same field locations for all instructions?

- 5.4** [M] At some point in the execution of a program, registers R4, R6, and R7 contain the values 1000, 7500, and 2500, respectively. Show the contents of registers RA, RB, RZ,

RY, and R6 in Figure 5.8 during steps 3 to 5 as the instruction

Subtract R6, R4, R7

is fetched and executed, and also during step 1 of the instruction that is fetched next.

5.5 [M] The instruction

And R4, R4, R8

is stored in location 0x37C00 in the memory. At the time this instruction is fetched, registers R4 and R8 contain the values 0x1000 and 0xB2500, respectively. Give the values in registers PC, R4, RA, RM, RZ, and RY of Figures 5.8 and 5.10 in each clock cycle as this instruction is executed, and also in the first clock cycle of the next instruction.

5.6 [D] Modify the expressions given in Example 5.3 to compare two, 4-bit, signed numbers in 2's-complement representation.

5.7 [E] The subroutine-call instructions described in Chapter 2 always use the same general-purpose register, LINK, to store the return address. Hence, the return register address is not included in the instruction. However, the address LINK is included in bits IR_{31-27} of subroutine-return instructions (see Section 5.4.1 and Example 5.4). Why are the two instructions treated differently?

5.8 [M] Give the execution sequence for the Return-from-interrupt instruction for a processor that has the interrupt structure given in Example 5.5. Assume that the address of register IRA is given in bits IR_{31-27} of the instruction.

5.9 [D] Consider an instruction set in which instruction encoding is such that register addresses for different instructions are not always in the same bit locations. What effect would that have on the execution steps of the instructions? What would you do to maintain a five-step execution sequence in this case? Assume the same hardware structure as in Figure 5.8.

5.10 [M] Assume that immediate operands occupy bits IR_{21-6} of the instruction. The immediate value is sign-extended to 32 bits in arithmetic instructions, such as Add, and padded with zeros in logic instructions, such as Or. Design a suitable implementation for the Immediate block in Figure 5.9.

5.11 [M] A RISC processor that uses the five-step sequence in Figure 5.4 is driven by a 1-GHz clock. Instruction statistics in a large program are as follows:

Branch	20%
Load	20%
Store	10%
Computational instructions	50%

Estimate the rate of instruction execution in each of the following cases:

(a) Access to the memory is always completed in 1 clock cycle.

(b) 90% of instruction fetch operations are completed in one clock cycle and 10% are completed in 4 clock cycles. On average, access to the data operands of a Load or Store instruction is completed in 3 clock cycles.

- 5.12** [E] The execution of computational instructions follows the pattern given in Figure 5.11 for the Add instruction, in which no processing actions are performed in step 4. Consider a program that has the instruction statistics given in Problem 5.11. Estimate the increase in instruction execution rate if this step is eliminated, assuming that all execution steps are completed in one clock cycle.
- 5.13** [D] Figure 5.16 shows that step 3 of a conditional branch instruction may result in a new value being loaded into the PC. In pipelined processors, it is desirable to determine the outcome of a conditional branch as early as possible in the execution sequence. What hardware changes would be needed to make it possible to move the actions in step 3 to step 2? Examine all the actions involved in these two steps and show which actions can be carried out in parallel and which must be completed sequentially.
- 5.14** [M] The instructions of a computer are encoded as shown in Figure 5.12. When an immediate value is given in an instruction, it has to be extended to a 32-bit value. Assume that the immediate value is used in three different ways:
- (a) A 16-bit value is sign-extended for use in arithmetic operations.
 - (b) A 16-bit value is padded with zeros to the left for use in logic operations.
 - (c) A 26-bit value is padded with 2 zeros to the right and the 4 high-order bits of the PC are appended to the left for use in subroutine-call instructions.

Show an implementation for the Immediate block in Figure 5.19 that would perform the required extensions.

- 5.15** [E] We have seen how all RISC-style instructions can be executed using the steps in Figure 5.4 on the multi-stage hardware of Figure 5.8. Autoincrement and Autodecrement addressing modes are not included in RISC-style instruction sets. Explain why the instruction

Load R3, (R5)+

cannot be executed on the hardware in Figure 5.8.

- 5.16** [E] Section 2.9 describes how the two instructions Or and OrHigh can be used to load a 32-bit value into a register. What additional functionality is needed in the processor's datapath to implement the OrHigh instruction? Give the sequence of actions needed to fetch and execute the instruction.
- 5.17** [E] During step 1 of instruction processing, a memory Read operation is started to fetch an instruction at location 0x46000. However, as the instruction is not found in the cache, the Read operation is delayed, and the MFC signal does not become active until the fourth clock cycle. Assume that the delay is handled as described in Section 5.6.2. Show the contents of the PC during each of the four clock cycles of step 1, and also during step 2.
- 5.18** [M] Give the sequence of steps needed to fetch and execute the two special instructions

MoveControl Ri, IPS

and

MoveControl IPS, Ri

used in Example 5.6.

- 5.19** [D] What are the essential differences between the hardware structures in Figures 5.8 and 5.22? Illustrate your answer by identifying the difficulties that would be encountered if one attempts to execute the instruction

Subtract LOC, R5

on the hardware in Figure 5.8. This instruction performs the operation

$$\text{LOC} \leftarrow [\text{LOC}] - [\text{R5}]$$

where LOC is a memory location whose address is given as the second word of a two-word instruction.

- 5.20** [M] Consider the actions needed to execute the instructions given in Section 5.4.1. Derive the logic expressions to generate the signals C_select, MA_select, and Y_select in Figures 5.18 and 5.19 for these instructions.

- 5.21** [E] Why is it necessary to include both WMFC and MFC in the logic expression for Counter_enable given in Section 5.6.2?

- 5.22** [E] Explain what would happen if the MFC variable is omitted from the expression for PC_enable given in Section 5.6.2.

- 5.23** [M] Derive the logic expressions to generate the signals PC_select and INC_select shown in Figure 5.20, taking into account the actions needed when executing the following instructions:

Branch: All branch instructions, with a 16-bit branch offset given in the instruction

Call_register: A subroutine-call instruction with the subroutine address given in a general-purpose register

Other: All other instructions that do not involve branching

- 5.24** [M] A microprogrammed processor has the following parameters. Generating the starting address of the microinstruction of an instruction takes 2.1 ns, and reading a microinstruction from the control store takes 1.5 ns. Performing an operation in the ALU requires a maximum of 2.2 ns, and access to the cache memory requires 1.7 ns. Assume that all instructions and data are in the cache.

(a) Determine the minimum time needed for each of the steps in Figure 5.26.

(b) Ignoring all other delays, what is the minimum clock cycle that can be used for this processor?

- 5.25** [M] Give the sequence of steps needed to fetch and execute the instruction

Load R3, (R5)+

on the processor of Figure 5.24. Assume 32-bit operands.

- 5.26** [M] Consider a CISC-style processor that saves the return address of a subroutine on the processor stack instead of in the predefined register LINK. Give the sequence of actions needed to execute a Call_Register instruction on the processor of Figure 5.24.